

博士后出站考核汇报

大模型基础设施 关键技术与工程实践

MoE 专家负载均衡、数据工程平台与推理网关

汇报人：陶壮

汇报日期：2026年9月18日

我的工作沿**三条项目线**逐步展开

博士后周期：2025.01—2027.01 本次汇报进展截至 2026.09

2025.01—03

开题探索与方向确认

确定 MoE 专家亲和性驱动的
推理优化为探索方向

2026.03—04

通信与独立 SDK 延续

新 HCCL context 支持
延续到 Omni-EPLB SDK

2025.03—06

C++ 核心开发与算法迭代

交换、冗余与调整收益
Pattern 工具、UT 和算法验证

2026.05—09

DataInfra 平台开发

Extensions、GMS、frontend
Data Agent 与配套验证

2025.07—12

开源集成与技术输出

工具、动态更新、模型适配
技术报告与 OmniInfer 团队论文

2026.08—09

LiteLLM 网关工程

会话亲和与负载感知路由
指标修复、独立 Redis 刷新

研究主线是 OmniPlacement，另外两项工作分别面向数据平台和推理服务入口

围绕三类问题，介绍这两年的工作

01 OmniInfer: OmniPlacement 研究与实现

专家为什么不均衡，怎样排布，怎样在线调整

02 DataInfra: 数据工程平台开发

先看平台，再讲 Extensions、GMS、frontend 和 Data Agent

03 Omni-ELB: LiteLLM 网关与请求路由

请求怎样选集群，负载状态怎样刷新和维护

开题后的研究重点落在了专家**负载均衡**

开题方向	实际开展的工作	目前进展
专家亲和性与通信优化	专家冷热建模、重排、冗余与拓扑排布	重点落在专家负载均衡
完善推理规划组件	Pattern 工具、动态调整、通信与适配	完成核心实现并持续迭代
技术报告与论文	OmniPlacement 技术报告、OmniInfer 团队论文	方法总结与系统实验
专家预读取与语义理解	暂未形成独立研究成果	留作后续探索

研究目标始终是优化推理，实际工作逐步聚焦到专家排布和动态均衡

01

OmniInfer: OmniPlacement

围绕专家冷热不均，研究排布方法，并把它接入实际推理框架

项目仓库

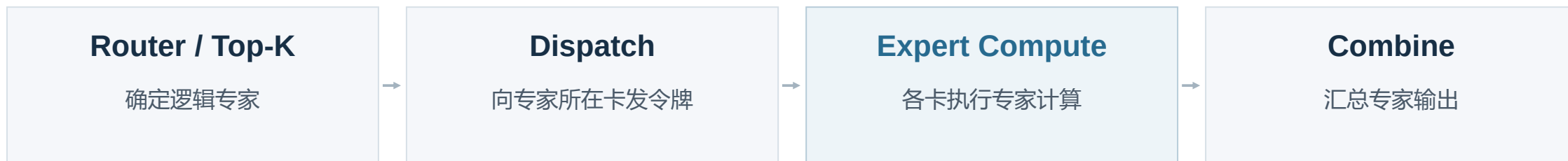
<https://gitee.com/omniai/omniinfer>

在 OmniInfer 中，我主要做**专家负载均衡**



我的工作集中在图中的 OmniPlacement 模块：排布、动态更新，以及接入这些能力的工具

专家分得均匀，卡上的**负载**仍可能不均



有些专家更忙

专家数量均匀分配，
不代表计算量均匀分配

不同层的热点不同

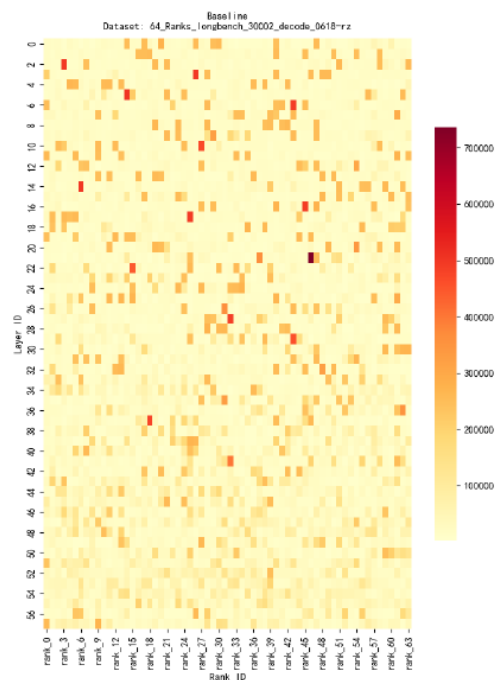
不同 MoE 层的激活分布不同，
统一冗余预算难以适配

调整也有成本

额外副本占用显存，
跨卡通信和在线更新也有代价

排布要跟着实际负载走，不能只看每张卡放了多少个专家

两种办法：挪开**热点**，增加热点副本



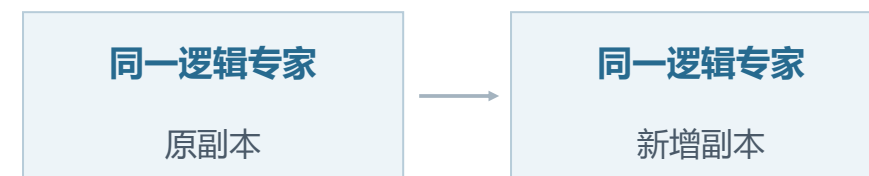
热点随层、随卡变化

同样数量的专家，
可能带来不同的计算压力。

重排：把同一卡上的热点分开



冗余：给热门专家多一份副本



先把热点分散开；如果一个专家仍然太忙，再给它增加副本

冗余副本有限，先给更需要的层和专家

01

先决定哪些层需要冗余

对高负载层启用额外副本
其他层保留基础专家部署

02

再决定给哪些专家加副本

用堆维护每个副本的估计负载
优先拆分热点专家的压力

03

最后落到设备与主机上

将专家分配到设备槽位
结合主机拓扑分散高负载卡

副本估计负载 = 专家激活量 / 副本数

这里先按副本均分负载估算
实际执行还受路由与通信影响

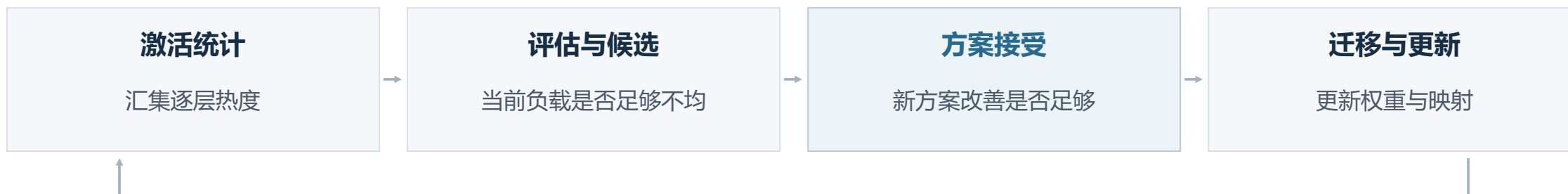
把有限的显存预算用在热点上，同时考虑设备槽位和主机拓扑

负载变了要调整，但**不能**频繁地搬专家

离线初始化



在线更新



先判断是否需要调整，再判断新方案是否值得替换旧方案

开源之前，我已经在做 C++ 核心开发

46

内部 MR 创建记录

内部研发阶段：2025 年 3 月至 7 月

把原型补成可用的 C++ 实现

核心类与接口、参数约束
热点排序、rank 检查
内存拷贝异常处理、代码注释

一边实现，一边明确研究问题

多冗余、热冷卡分配与调整收益
静态 / 动态 Pattern 怎样配合
不同层需要多少冗余

2025 年 4 月开始集中讨论这些问题

这段内部研发，逐步形成了后续专家均衡的核心实现和研究思路

算法经过**多轮迭代**，工具和测试同步跟上

方法不断调整

2025.05

swap optimizer、简单权重交换
单 rank 交换约束及配套 UT

2025.06 上旬

冗余 / 非冗余统一处理
Hungarian 输出确定性、算法验证

2025.06 中下旬

greedy / bucket 方法迭代
停止条件、递归与优先级排序、批次排序

工具与测试同步完善

生成与接入

Pattern 生成流水线与基础配置
基准 Pattern、路径与部署工具

测试与调试

C++ / Python UT、测试耗时优化
调试开关、算法验证与异常修复

分析与交付

负载不均衡可视化与使用文档
比较候选方案，定位问题并联动

每轮算法修改，都需要配套的测试、调试工具和可比较的输出

开源以后，继续解决接入和运行中的问题

2025.07—08	Pattern 工具与部署	数据整理、生成与分析流水线 分层冗余、共享专家场景支持
2025.08—11	运行机制与适配	批量发送、动态侧算法支持 LongCat 零专家与 Qwen 适配
2025.09—12	测试与配置	UT 构建和硬件测试入口 CLI 配置、YAML 更新逻辑
2026.03—04	通信能力延续	新 HCCL context 支持 在独立 Omni-EPLB 仓库延续

算法之外，配置、接口、测试和模型差异也需要逐项处理

开源阶段持续投入实现、重构与测试

13

现存历史中的已合入 PR

7,820

累计新增行

9,993

累计删除行

开发内容	具体工作
排布工具	Pattern 流水线、CLI 配置、YAML 更新和测试
运行与适配	分层冗余、共享专家、动态更新和模型适配
SDK 延续	新 HCCL context 支持，延续至 Omni-EPLB

2025.07—2026.03 · 代码变更含源码、测试、配置和文档

代码改动主要围绕排布工具、运行适配和测试展开

独立为 SDK 后，排布能力可以**单独接入**

早期：集成于 OmniInfer

随推理框架交付排布与动态均衡

omniinfer/omni/accelerators/placement

延续已有排布与运行能力

Pattern 生成、分层冗余与动态均衡

批量发送、多模型适配与 UT

当前：独立 Omni-EPLB

单独维护接口、配置与运行时组件

omni-eplb/omni_placement

通过接口接入不同场景

Python 规划与 placement_handler

C++ 均衡器、映射及分布式操作

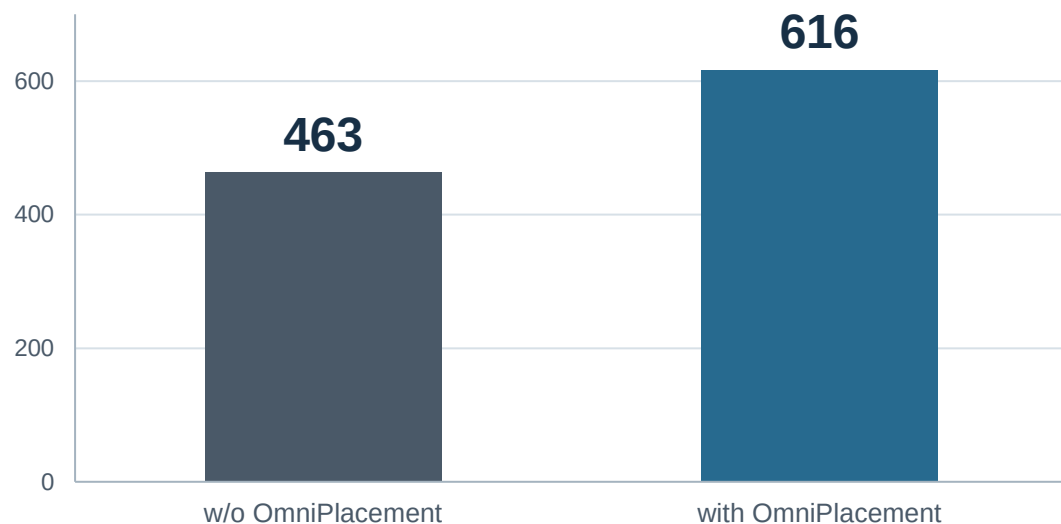
<https://gitee.com/omniai/omni-eplb>

核心能力继续演进，接入方式从框架内模块扩展到独立 SDK

开启 OmniPlacement 后，吞吐提升 33.0%

DeepSeek-R1 INT8, 10 节点 Ascend 910C, 6P8-1D32; 平均输入 3.5K、输出 1K tokens

QPM 对比 (论文原值)



每输出 token 延迟 TPOT

降低 26.2%

由 65 ms 降至 48 ms

端到端延迟 E2E

降低 24.1%

由 78.025 s 降至 59.206 s

吞吐和生成阶段延迟改善，但首 token 等待更长：TTFT 从 1.342 s 增至 2.292 s

02

DataInfra：数据工程平台

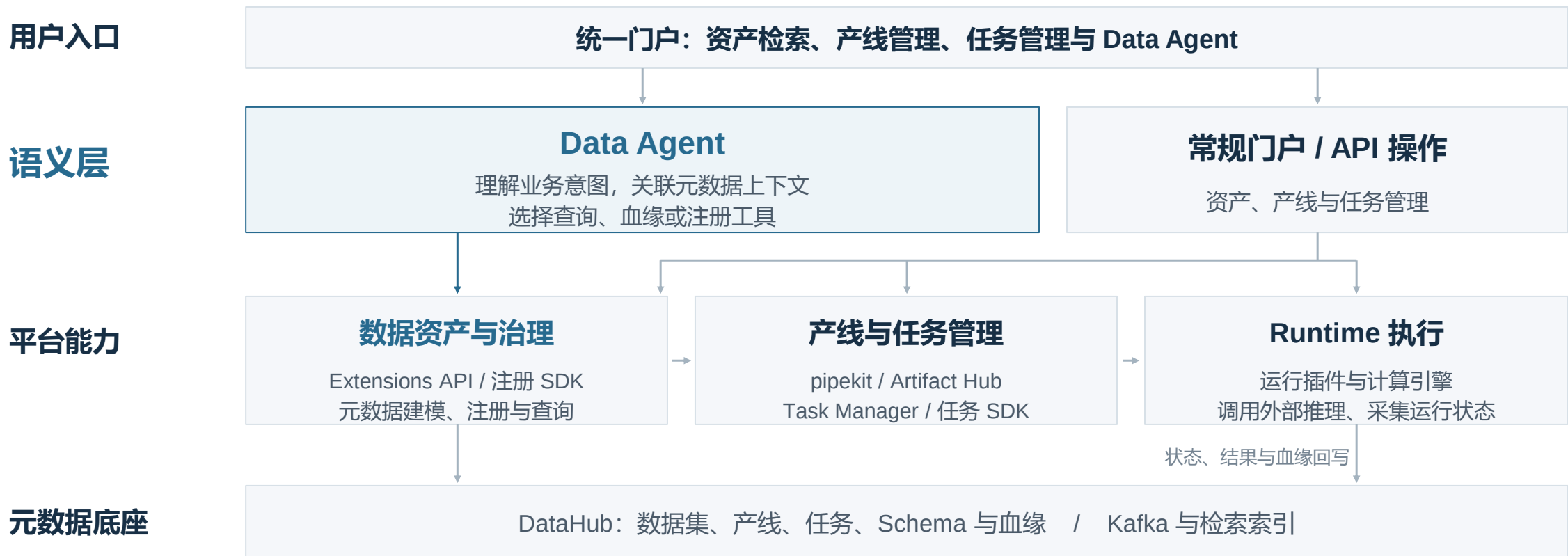
这部分工作关注数据怎样登记、管理，以及怎样让用户方便地找到和使用

内部项目仓库（CodeHub）

https://codehub-g.huawei.com/AI_Infra/AI_Infra_DataFlow/data_infra_platform/home

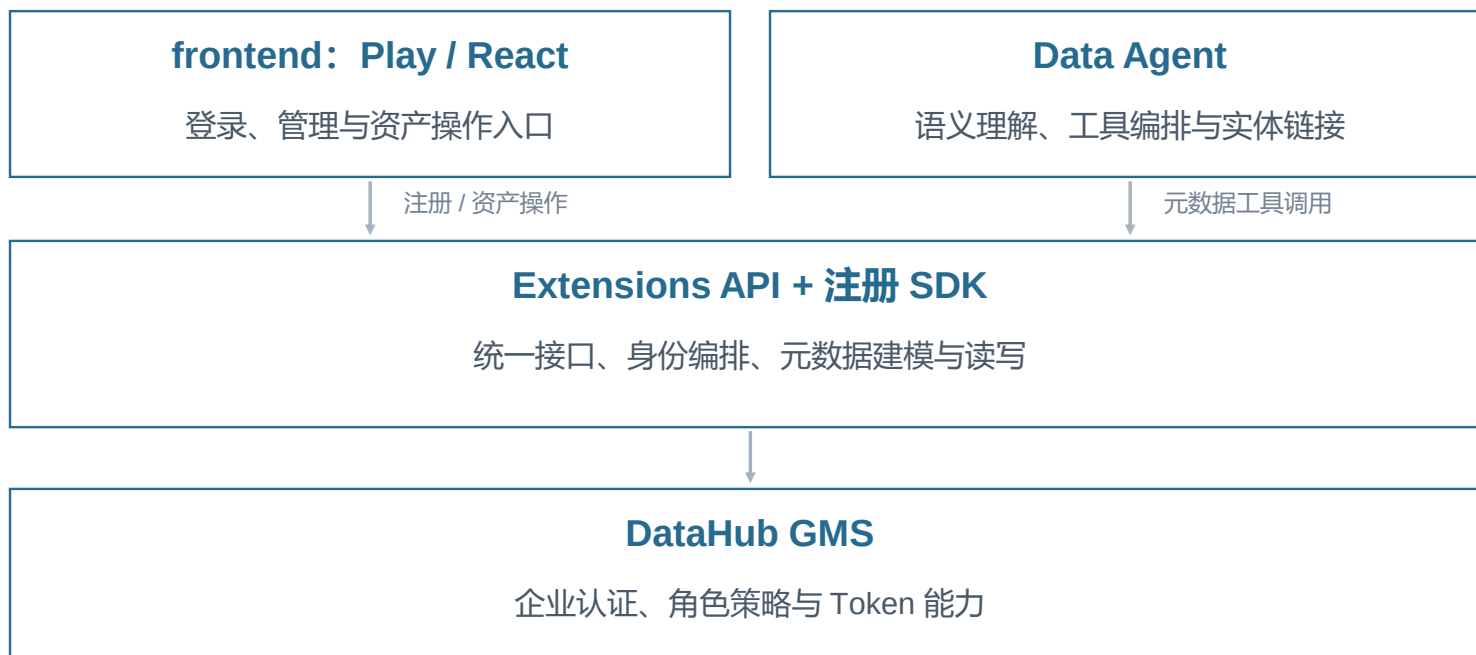
Data Agent 是 DataInfra 面向用户的语义层

承接业务问题，理解数据含义与关系，再映射到元数据工具和平台接口



平台依赖：对象存储、容器资源、分布式计算、外部推理，以及统一身份与监控能力

我主要打通**元数据**和用户入口之间的链路



Extensions 重构

拆分网关与 SDK，形成复用接口

GMS 改造

SSO、访问准入、角色与 Token

frontend 集成

登录、权限与资产操作页面

Data Agent 语义层

语义理解、对话工具与实体跳转

登录与 GraphQL 也可直接访问 GMS，注册和工具调用经过扩展层

沿着用户入口往下看，四块工作最终汇到同一条元数据链路

拆开扩展服务，让注册能力可以被多处复用

原来：EntityBuilder

建模、查询、发送、API

GraphQL / REST / Kafka

集中在同一模块

建模和 I/O 混在一起

难以分别测试和复用

身份与权限处理也需要拆清楚



注册 SDK 内部：纯计算与 I/O 分离

实体层负责 URN / Aspect / 变更事件构建

服务层编排校验，数据访问与发布层处理 I/O

前端通过 API 使用能力，平台其他模块也可以直接调用 SDK

接入企业 SSO 后，还要把**身份和权限**接通

IDaaS 认证接入

认证器与 SSO 回调
账号创建和身份映射

访问准入与审计

工号 / 部门白名单
运行时管理与登录记录

角色与 Token 支持

Publisher 角色及策略
Token Aspect 与 GraphQL

一个接入难点：GMS 和 Play frontend 不共享会话

GMS / Jetty

完成 SSO 握手及准入校验

Play frontend / Netty

建立平台登录会话

React 门户

使用登录态访问平台能力

登录成功只是第一步，后续每次访问仍要带上身份并完成权限校验

后台能力做好后，还要让用户能直接使用

Play frontend：接好登录态和凭证

SSO 回调与平台会话建立

NeverTokenManager 自动发放凭证

按登录态获取 Token，衔接 API 调用

React：提供用户能操作的页面

白名单 / 部门授权管理及表格导入

注册 YAML、记录预览、角色选择

Token 管理、配额页与 Data Agent 入口



页面给出操作入口，凭证随请求传递，最终由服务端判断能不能执行

Data Agent 把用户问题映射到具体资产与操作

语义层说明数据是什么、彼此怎样关联，以及可以怎样使用

问题示例：这个数据集从哪里来？包含哪些字段？



语义上下文：领域与描述、标签与字段、数据血缘，以及注册规则

语义层的架构职责

把业务问题对应到数据实体与关系
将查询与注册约束带入工具调用

支撑这一层的工程实现

多轮对话、工具编排、查询与注册
实体链接、流式展示与回归测试

DataInfra 的 19 项 PR，覆盖平台接入到 Agent



开发内容	具体工作
企业接入	SSO、访问准入、角色权限和 Token
平台与 Agent	Extensions / 注册 SDK、GMS、frontend、Data Agent
实现与重构	注册逻辑拆分、SDK 复用、前后端接口衔接

2026.05—2026.09 · 代码变更含源码、测试、配置和文档

工作重点从企业接入、扩展重构，逐步延伸到资产操作和 Data Agent

03

Omni-ELB: LiteLLM 网关

请求进入网关后，需要结合会话和负载，选择合适的模型集群

项目仓库

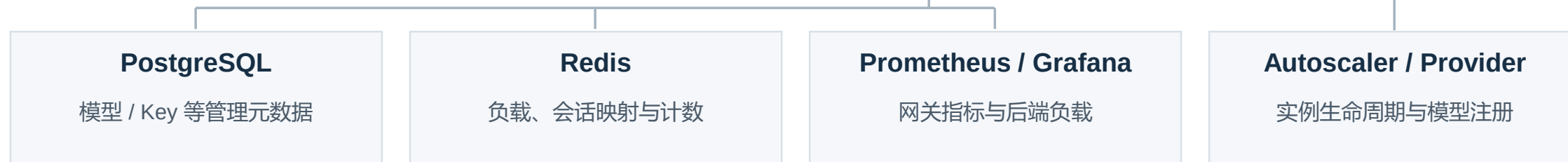
<https://gitee.com/omniai/omni-elb>

一次推理请求，怎样经过网关到达模型集群

业务请求主链路



共享状态、可观测与实例管理

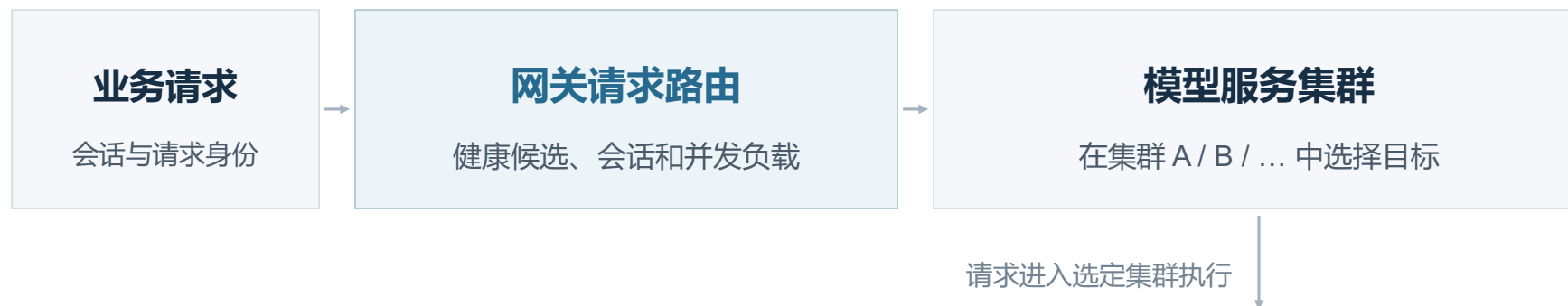


网关把业务请求接进来，再决定交给哪个模型集群处理

请求路由把负载均衡扩展到模型服务层

模型服务级

请求交给哪个集群



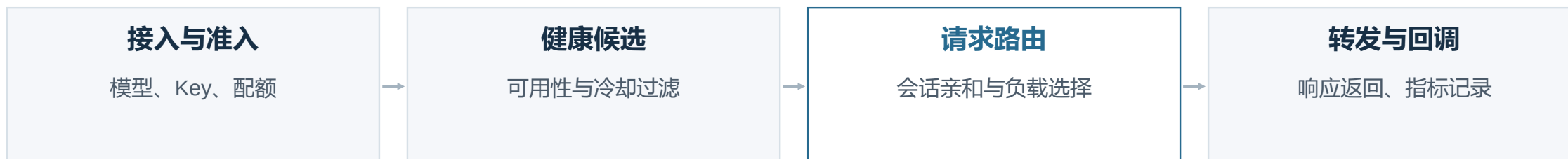
模型执行级

专家计算怎样分到卡上



请求路由调节集群间的请求分布，EPLB 调节模型内部的专家计算分布

可用集群不止一个，路由还要做一次选择



选集群时，先看身份和状态

这个请求来自谁

user_id / PipelineKey / APIKeyHash

当前有哪些状态

Redis 会话映射与集群 inflight

最后发到哪里

保留健康会话，或选择新的目标集群

路由同时看请求身份、会话记录和当前负载，再决定请求发往哪里

老会话尽量不搬，新会话优先选**低负载集群**

负载状态链路



不同请求，使用不同的选择规则

请求类型	选择规则
user_id	缓存命中且集群健康：复用会话集群 否则在最低负载 10% 候选中随机选取
PipelineKey	返回健康候选，由 LiteLLM 默认策略选择
APIKeyHash	按稳定哈希选择集群，作为兼容路径

无可用负载时按哈希兜底；选定后缓存会话映射

优先保持健康会话的稳定，新会话再根据当前负载分散到不同集群

路由要看准负载，状态刷新和计数就要准确

Redis 刷新：把配置和状态接起来

遇到的问题	处理方式
多份脚本分散维护	整合为独立生成脚本和运行入口
模型配置依赖手工维护	从 /model/info 读取模型与集群映射
集群名变化后难以定位	对照配置与监控，提示缺失或疑似改名
配置接口可能不可用	保留原有配置作为回退

Prometheus：修复请求重复统计

PipelineKey 请求出现重复计数
调整日志过滤和计数路径
修正监控中这一路径的数据

刷新链路负责把状态送过来，计数修复保证这个状态反映真实请求

项目信息与成果展示

OmniPlacement 已随 OmniInfer 开源，相关能力在 Omni-EPLB 中持续演进

技术报告

OmniPlacement

昇腾超大规模 MoE 模型推理
负载均衡技术报告

昇腾超大规模 MoE 模型推理部署
技术开源周发布

论文 / SESAME'26 @ EuroSys

OmniInfer

System-Wide Acceleration Techniques
for Optimizing LLM Serving
Throughput and Latency

共同作者: Zhuang Tao

预印本首发: 2025.11.27

arxiv.org/abs/2511.22481

3 项

专利申请

1 项

技术发布

8 项

成果转化

回看这两年，重点是把方法做进可用的系统

项目	我重点做了什么	形成了什么
OmniInfer / OmniPlacement	专家排布、动态均衡、通信和适配测试	算法与工具实现 团队消融 QPM +33.0%，同时存在 TTFT 权衡
DataInfra	Extensions 重构、GMS / frontend、Data Agent	复用的元数据 SDK 企业接入、门户和资产工具
Omni-ELB / LiteLLM	参与路由设计 实现状态刷新与计数修复	会话与负载相结合的路由 独立刷新脚本、修正后的指标链路

下一步继续做：补充不同负载下的评估，细化 Agent 工具权限，完善线上状态观测

算法需要进入系统，平台能力需要走到用户入口，路由决策需要可靠的状态输入

感谢一路指导和支持我的领导、导师与同事

感谢领导和导师在选题、研究和项目推进中的指导

特别感谢 **Ken Zhang** 老师
指导我从工程实践中提炼研究问题，并将理论思考落实到系统实现

感谢一起做项目的同事
一起讨论方案、评审代码、排查问题，把工作一点点推进

感谢各位专家的时间和建议，请批评指正

陶壮 2026年9月18日

附录 A：怎样估算每张卡上的专家负载

变量定义

$a(l,e)$: 专家激活量

$r(l,e)$: 专家物理副本数

$x(l,e,d)$: 专家是否部署在设备 d

先估计每张卡要处理多少负载

$$L(l,d) = \sum_e x(l,e,d) \cdot a(l,e) / r(l,e)$$

再让最忙的卡尽量轻一些

$$\min \max_d L(l,d)$$

约束：专家完整覆盖、每卡槽位、副本上限与显存预算

适用前提：副本可分摊请求。通信代价和更新代价需单独建模、测量

附录 B：完整性能对比与测试条件

指标	w/o OmniPlacement	with OmniPlacement	相对变化
QPM	463	616	+33.0%
TPOT (ms)	65	48	-26.2%
p99 TPOT (ms)	68	50	-26.5%
TTFT (s)	1.342	2.292	+70.8%
p99 TTFT (s)	2.419	4.198	+73.5%
E2E (s)	78.025	59.206	-24.1%
p99 E2E (s)	196.226	148.364	-24.4%
OTT (tok/s)	45,372	60,575	+33.5%
TTT (tok/s)	169,083	225,546	+33.4%

论文报告的测试条件

DeepSeek-R1, INT8
 Ascend 910C, 6P8-1D32
 平均输入 3.5K、输出 1K tokens
 合计长度小于 16K，长尾分布
 补充完成请求，保持固定并发

看结果时需要一起考虑

其余组件保持不变
 TTFT 上升，生成阶段延迟下降
 QPM 沿用论文原值，未换算单位

相对变化以未启用 OmniPlacement 的结果为基准，其他组件保持不变

附录 C：各模块的具体工作

模块	我的工作	代码与文档
OmniPlacement	C++ 优化器、Pattern、动态均衡、通信及适配	旧 placement / 新 omni-eplb 内部历史与开源提交
Extensions	统一 API、注册 SDK、权限与 Domain 校验	Extensions / 注册 SDK 设计文档
GMS / frontend	SSO、访问名单、Publisher、Token 与门户	认证、权限和门户相关模块
Data Agent	对话 SDK、元数据工具、实体链接与配置	datahub_chat / ChatPage
Omni-ELB 路由	与同事共同设计会话亲和及负载感知算法	SEG 报告 §2.1 api_key_router.py
Omni-ELB 状态	独立 Redis 刷新、PipelineKey 计数修复	独立刷新脚本 !100 / !98